indices need to be non-negative integers, we simply round the value to the nearest whole number with `np.round(...)`. We also need to clip any values outside the minimum and maximum support range. For example, if the observed $r_t = -2$ then $b_j = \frac{-2-(-10)}{0.4} = \frac{-2+10}{0.4} = 20$. You can see that the support element with index 20 is –2, which in this case exactly corresponds to the observed reward (no rounding needed). We can then find the corresponding probability for the –2 support element using the index.

Once we find the index value of the support element corresponding to the observed reward, we want to redistribute some of the probability mass to that support and the nearby support elements. We have to take care that the final probability distribution is a real distribution and sums to 1. We will simply take some of the probability mass from the neighbors on the left and right and add it to the element that corresponds to the observed reward. Then those nearest neighbors will steal some probability mass from their nearest neighbor, and so on, as shown in figure 7.11. The amount of probability mass stolen will get exponentially smaller the farther we go from the observed reward.

Figure 7.11. The `update_dist` function redistributes probability from neighbors toward the observed reward value.